

ASSIGNMENT 3

Statistics for Machine Learning: Titanic Disaster

Submitted By:	Abhiney Kumar	Submitted To:	Dr. Ashok Gopalakrishnan
Enrollment No:	198409	Subject:	Statistics for Machine Learning
Email:	abhineyk.ec.24@nitj.ac.in	Date:	20 June 2026

Question 1: What have you learnt from the Titanic Disaster?

Beyond its historical tragedy, the 1912 Titanic disaster serves as a foundational case study for data-driven risk management and statistical modeling. From a data science perspective, the dataset highlights how heavily **societal biases** influence historical data. The stark contrast in survival rates across socioeconomic classes (**Pclass 1, 2, 3**) and genders proves that unadjusted models will simply inherit and amplify systemic human biases rather than uncovering objective truths.

The dataset also emphasizes the critical role of **feature engineering and selection**. Core variables such as **Age**, **Pclass**, **Sex**, **Fare**, and **SibSp** yield massive predictive power, proving the real-world machine learning principle that model accuracy relies heavily on selecting highly informative features rather than just dumping raw data into an algorithm. Furthermore, dealing with the widespread missing values in the **Age**, **Cabin**, and **Embarked** columns demonstrates that handling missing data—whether through mean/median imputation or row dropping—is a core preprocessing hurdle that directly dictates model accuracy.

Finally, the data serves as a classic warning that **correlation does not imply causation**. While a higher fare strongly correlates with survival, the monetary transaction itself saved no one; it was merely a proxy for physical proximity to lifeboats, structural cabin location, and priority boarding. In engineering terms, the disaster represents a cascading failure of risk management—where insufficient lifeboats, ignored warnings, and poor training combined catastrophically—reiterating that data scientists must account for safety margins, missing data, and the real-world ethical implications of their predictive models.

Question 2: Distractions are dangerous. Elaborate.

The maritime errors that caused the Titanic to strike an iceberg map directly onto the core failures encountered when training machine learning models. Historically, the ship's wireless operators were overwhelmed by arbitrary passenger messages, causing them to miss and fail to relay urgent iceberg warnings to the bridge. In data science, this mirrors the **curse of dimensionality**, where including noisy, irrelevant features (such as **Passenger ID** or **Ticket Number**) distracts the model from learning the actual underlying signal, dragging down overall accuracy.

Similarly, just as the lookouts were severely handicapped by a lack of binoculars or the captain was distracted by corporate pressure to maintain speed, models frequently fall victim to **overfitting**. When a model focuses too

closely on the noise within its training set, it becomes highly distracted by localized anomalies and entirely fails to generalize to unseen test data.

Ignoring outliers in data is just as fatal as dismissing physical iceberg warnings. If anomalous spikes are ignored during preprocessing, the resulting production models can deliver dangerously flawed predictions. This is worsened when data science teams attempt to simultaneously optimize too many conflicting metrics (**accuracy, precision, recall, F1**) without a clear primary objective. This systemic lack of prioritization results in poorly optimized models that underperform across the board.

Lastly, modern deployment environments frequently suffer from **alert fatigue**. When monitoring tools flood engineering teams with low-priority warnings, critical system failures or model drift go completely unnoticed until a catastrophic crash occurs. Ultimately, eliminating unnecessary structural complexity and feature noise is a strict discipline required to build reliable, high-stakes engineering systems.

Question 3: Stakeholders should be kept informed. Yes/No?

Answer: YES — Absolutely and unequivocally.

Transparent communication with stakeholders is an absolute ethical and operational necessity in engineering projects. A breakdown in communication on the Titanic led to avoidable loss of life: passengers were left in the dark about the true severity of the damage (causing many to refuse early lifeboat boarding), third-class passengers were completely cut off from evacuation routes, and the White Star Line actively downplayed the emergency to shield its corporate reputation. If clear, audited communication protocols had existed to link the crew, passengers, and rescue operations, the survival rate would have been significantly higher.

Translating this to modern AI infrastructure, business stakeholders must explicitly understand a model's edge cases, operational boundaries, and failure modes to prevent flawed executive decisions. From a user perspective, end-users have a right to know when automated AI systems are handling high-stakes determinations like loan approvals or medical diagnostics.

Furthermore, growing compliance landscapes (such as the EU AI Act or GDPR) mandate transparency reports for regulatory bodies. When technical teams intentionally conceal poor model performance or data decay from management, they accumulate severe technical debt, setting up the organization for catastrophic failures once the system goes live in production.

Question 4: Traceability is essential. What do you think?

Traceability is the absolute backbone of accountability, safety, and rigorous scientific engineering. The chaos of the Titanic evacuation serves as an enduring warning of what happens when a system lacks clear tracking. Because incoming wireless ice warnings were never systematically logged or escalated, they could not be audited or acted upon. The high-stakes decision to maintain maximum speed through hazardous waters was never recorded, destroying post-disaster accountability. Even the lifeboats were launched half-empty because there was no traceable system to coordinate passenger assignments.

In machine learning workflows, this chaos is managed via strict MLOps principles:

Model Versioning & Data Lineage: Every deployment must be tracked back to its exact code commit, hyperparameter configuration, and data snapshot. If a production model outputs a flawed prediction, engineers must be able to audit its exact data lineage—knowing where the data originated and how it was transformed.

Experiment Tracking: Relying on tools like MLflow or Weights & Biases allows teams to trace the progression of experimental metrics across iterations, ensuring that successful outcomes are actually reproducible rather than lucky anomalies.

Regulatory Audits: In strictly regulated domains like healthcare and finance, comprehensive audit trails are a mandatory legal requirement. Without robust traceability, an ML system becomes an unprovable, untrustworthy black box.

Question 5: Documentation may have lasting benefits. True or False and Why?

Answer: TRUE.

Comprehensive documentation transforms isolated technical efforts into enduring organizational knowledge. The only reason the Titanic tragedy was able to fundamentally reform global maritime law, ship engineering, and safety protocols was because investigators had access to extensive documentation, including surviving wireless logs, passenger manifests, and detailed testimonies. This documentation directly birthed the **International Convention for the Safety of Life at Sea (SOLAS) in 1914**. Over a century later, SOLAS remains in force worldwide, mandating continuous radio watches and sufficient lifeboat capacities—a clear proof of documentation providing multi-generational benefits. Conversely, the unrecorded bridge conversations and undocumented orders remain historical mysteries that permanently block our understanding of the final errors.

In data science, skipping documentation creates immense institutional vulnerability:

Knowledge Preservation & Onboarding: When an engineer leaves an organization, a well-documented pipeline ensures their work survives their departure, allowing new team members to onboard via design logs rather than costly trial and error.

Compliance & Reproducibility: Landmark regulations like the GDPR, HIPAA, and the EU AI Act make undocumented AI decision-making a severe legal liability. Proper documentation guarantees reproducibility, allowing future researchers to safely verify and build upon past experiments.

Model Cards: Implementing modern artifacts like "Model Cards" and "Datasheets" establishes responsible AI deployment by explicitly detailing what a model can and cannot do. Ultimately, while code runs software, documentation is what scales knowledge across an entire enterprise.

Practical Exercises: Python Implementation

The following implementations demonstrate data preprocessing, model training, and performance evaluation using the Titanic dataset.

Exercise 1: Linear Regression — Continuous Fare Prediction

This model predicts the continuous numeric value of passenger fares based on age, passenger class, and family size (*SibSp*).

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
titanic = pd.read_csv('titanic.csv')
X = titanic[['Age', 'Pclass', 'SibSp']]
y = titanic['Fare']

# Handle missing data using mean imputation
X = X.fillna(X.mean())
y = y.fillna(y.mean())

# Split into 80/20 train/test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Model Training
model = LinearRegression()
model.fit(X_train, y_train)

# Evaluation
y_pred = model.predict(X_test)
print('Mean Squared Error:', mean_squared_error(y_test, y_pred))
print('R2 Score:', r2_score(y_test, y_pred))

# Inferencing new test case (Age 25, Pclass 2, SibSp 1)
new_passenger = [[25, 2, 1]]
print('Predicted Fare:', model.predict(new_passenger)[0])
```

Key Analytical Insight: The resulting R^2 score measures the proportion of variance explained by the features. A low R^2 indicates that passenger fare cannot be modeled purely by age and class alone; it depends on more nuanced features such as cabin allocations or embarkation ports.

Exercise 2: Logistic Regression — Binary Survival Classification

This implementation handles a discrete binary classification task ($0 = \text{No}$, $1 = \text{Yes}$) to predict passenger survival.

```

import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Data Loading and Encoding
titanic = pd.read_csv('titanic.csv')
X = titanic[['Age', 'Pclass', 'Sex']]
y = titanic['Survived']

# Preprocessing: Imputation and Binary Mapping
X['Age'] = X['Age'].fillna(X['Age'].mean())
X['Sex'] = X['Sex'].map({'male': 0, 'female': 1})

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Training Logistic Regression with adequate iterations
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# Evaluation
y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print('
Confusion Matrix:
', confusion_matrix(y_test, y_pred))
print('
Classification Report:
', classification_report(y_test, y_pred))

# Test Case: Female, Age 25, Pclass 2
new_passenger = [[25, 2, 1]]
result = model.predict(new_passenger)
print('
Predicted Survival:', 'Yes' if result[0] == 1 else 'No')

```

Key Analytical Insight: *Logistic regression maps linear features to a probability space between 0 and 1 via the sigmoid function. Analyzing the confusion matrix is vital here, as it highlights the operational trade-offs between critical false positives and false negatives.*

Exercise 3: Supervised vs. Unsupervised Learning Paradigms

This exercise runs a direct comparative pipeline, benchmarking a supervised Logistic Regression model against an unsupervised K-Means Clustering algorithm using the same underlying attributes.

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

titanic = pd.read_csv('titanic.csv')

# --- PART 1: SUPERVISED PIPELINE ---
X_sup = titanic[['Age', 'Pclass', 'Sex']]
y_sup = titanic['Survived']
X_sup['Age'] = X_sup['Age'].fillna(X_sup['Age'].mean())
X_sup['Sex'] = X_sup['Sex'].map({'male': 0, 'female': 1})

X_train, X_test, y_train, y_test = train_test_split(X_sup, y_sup, test_size=0.2,
random_state=42)

log_model = LogisticRegression(max_iter=200)
log_model.fit(X_train, y_train)
y_pred = log_model.predict(X_test)

print('=== Supervised Learning (Logistic Regression) ===')
print('Accuracy:', accuracy_score(y_test, y_pred))

# --- PART 2: UNSUPERVISED PIPELINE ---
# K-Means operates without utilizing target labels
X_cluster = titanic[['Age', 'Fare', 'Pclass']].fillna(titanic[['Age', 'Fare',
'Pclass']].mean())

kmeans = KMeans(n_clusters=3, random_state=42)
titanic['Cluster'] = kmeans.fit_predict(X_cluster)

print('
=== Unsupervised Learning (K-Means Clustering) ===')
print('Cluster Centers:
', kmeans.cluster_centers_)

# Plotting Cluster Districting
plt.figure(figsize=(8, 6))
plt.scatter(titanic['Age'], titanic['Fare'], c=titanic['Cluster'], cmap='viridis',
alpha=0.6)
plt.xlabel('Age')
plt.ylabel('Fare')
plt.title('Passenger Segmentation via K-Means Clustering')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

```